PHARMACY MANAGEMENT SYSTEM

Complete Technical & Product Catalog

Architecture · Features · Design · Operations · API

Version 1.0 · 2026-08-19

Repository: github.com/yasin6606/pharmacy-management-system

Author: yasin

Table of Contents

1. Executive Overview

The **Pharmacy Management System** is a full-stack, multi-branch platform designed for real-world pharmacy operations in Iran. It unifies inventory control (batch-level expiry), point-of-sale with Iranian Rial (IRR) pricing, patient insurance cost-sharing, card-terminal (POS) payment workflows, employee role-based access, loss reporting, purchasing hooks, reporting exports, and external integrations (Titak price feeds and Iranian social insurers).

The product prioritizes **operational safety** (stock locks, safe drug deletion, credit tracking), **observability** (structured logs, request IDs), and **deployability** (Docker Compose with Nginx edge, PostgreSQL, Redis).

Document purpose

This catalog is the authoritative technical and product reference: features, architecture, design systems, domain rules, APIs, security model, deployment topology, and operational guidance.

Primary stakeholders

- Pharmacy owners and managers (branch policy, franchise fees, integrations)
- Pharmacists and cashiers (sales, insurance, POS, inventory visibility)
- Accountants (credits, reports, revenue summaries)
- Engineers and operators (deploy, monitor, extend)

2. Product Goals & Domain Context

2.1 Goals

- Single system of record for drugs, batches, branches, sales, and staff
- Accurate stock under concurrent sales (pessimistic locking on batches)
- Native IRR presentation (whole rials; no USD residual in core UX)
- Insurance-aware checkout: only formulary-eligible drugs share cost with the insurer
- Configurable third-party keys (Titak, Tamin, Salamat, Mosalah) without redeploy
- Bilingual EN/FA and light/dark themes including unauthenticated screens
- Production-minded containers: one public port, internal data plane

2.2 Iranian pharmacy domain notes

Pharmacies commonly work with social insurance funds (Tamin, Salamat, Mosalah), electronic booklets/member IDs, and regulated price sources (e.g. Titak). Terminal payments require an acquirer-specific POS adapter. This system models those realities with explicit domain fields and settings, while treating live insurer claim APIs as contract-based adapters (keys are not shipped with the software).

3. Feature Catalog (Complete)

3.1 Core platform

- Multi-branch model: retail branches and warehouse flag
- First-run setup creates the initial manager account
- JWT authentication with session tracking (EmployeeSession)
- Employee branch assignment history
- Roles: junior, senior, manager, accountant

3.2 Inventory & catalog

- Drug master data: name, brand, company, entering date
- titakCode for external price sync; insuranceEligible + insuranceCode
- Paginated drug list with search (name/brand/company)
- Safe delete: blocked while any batch has remaining stock
- Batches: expiration, count, isOffer, purchasePrice, sellingPrice (IRR)
- Inter-branch stock transfer with movement audit trail
- Expiring-batch job / alerts threshold (e.g. 30 days)
- Catalog stats API for dashboard total drugs / units / batches

3.3 Sales & POS

- Multi-tab patient baskets (parallel customers at the counter)
- Payment methods: cash, transfer, POS, credit
- POS flow: initiate terminal session → cashier confirms approval/decline → complete sale with posReference
- Insurance providers: none, tamin, salamat, mosalah, other
- Per-line insuranceCoverageAmount and patientShareAmount
- Franchise fee optionally applied once per basket when branch.hasFranchise
- Sales summary endpoint (SQL aggregates; not page-limited)
- Sales records with branch filters for manager/accountant
- Credits: group by basket, search customer, mark paid

3.4 Operations

- Loss reports: create → review approve/reject workflow
- Purchasing module: suppliers, purchase orders, OCR client hook
- Reporting: filtered sales reports, CSV and PDF export
- Settings: franchise amount; integration secrets (masked); coverage %

3.5 UX / product design

- Glassmorphism design system (glass surfaces, medical teal & gold accents)
- Responsive dashboard, sidebar navigation, role-gated menu items
- i18n: English and Persian (Farsi) with always-locale URL prefix
- Theme toggle on login, setup, and authenticated shell
- Language switcher with Languages icon
- Severity-aware error toasts with optional request correlation id

3.6 Reliability & engineering quality

- Structured Winston logging; HTTP access logs with duration and requestId
- Unified AppError API responses; Zod validation mapping
- Frontend ApiError, client logger, toast deduplication
- Process handlers for uncaughtException / unhandledRejection
- Docker healthchecks and ordered service startup

4. System Architecture

4.1 Logical view

Clients (browser) communicate exclusively with **Nginx** on the published HTTP port. Nginx reverse-proxies `/api/*` and `/health` to the Express backend and all other paths to the Next.js frontend. PostgreSQL and Redis are reachable only on the internal Docker bridge network.

Request path

```
Browser → Nginx:80
  ■■ /api/v1/*  → backend:3001
  ■■ /health    → backend:3001
  ■■ /*         → frontend:3000
```

4.2 Component view

Component	Responsibility
Frontend (Next.js)	App Router UI, i18n, themes, API client, toasts
Backend (Express)	REST API, auth, domain services, integrations
PostgreSQL 16	System of record (TypeORM entities)
Redis 7	Shared rate-limit store (optional scale-out)
Nginx	Edge routing, future TLS termination

4.3 Modular backend

Backend modules under `backend/src/modules/*` encapsulate domain boundaries: auth, employees, branches, inventory, sales, settings, reporting, loss-reports, purchasing, setup, and integrations (titak, insurance, pos). Cross-cutting concerns live in `core/` (config, errors, logger, middleware, utils).

4.4 Frontend structure

Routes under `app/[locale]/(auth)` and `(dashboard)`. Shared UI in `components/ui`, forms in `components/forms`, contexts for auth/errors/sales tabs, hooks for API and roles, messages in en.json and fa.json.

5. Technology Stack

Layer	Technologies
Frontend	Next.js (App Router), React, TypeScript, Tailwind CSS, next-intl, next-themes, react-hook-form, Zod, Axios, Lucide
Backend	Node.js, Express, TypeScript, TypeORM, Awilix DI, Zod, Winston, bcryptjs, jsonwebtoken, ioredis, uuid
Database	PostgreSQL 16 Alpine
Cache	Redis 7 Alpine (AOF, memory cap)
Edge	Nginx 1.27 Alpine
Build	Webpack (backend), Next.js production build
Infra	Docker multi-stage images, Compose, optional Oracle Cloud ARM scripts

6. Backend Architecture & Modules

6.1 Cross-cutting core

Area	Key types / behavior
AppError	statusCode, code, details; factories badRequest/notFound/…
errorHandler	Zod, AppError, QueryFailed, JWT, SyntaxError → JSON
requestLogger	x-request-id, durationMs, status-tiered log level
logger	Winston; LOG_LEVEL; optional LOG_TO_FILES
auth + rbac	JWT bearer; requireRole middleware
validation	Zod schemas on write paths
pagination	Shared paginate helper (page, limit, totalPages)

6.2 Domain modules

Module	Capabilities
setup	Bootstrap first manager when no users exist
auth	Login, logout, me, change-password; sessions
employees	CRUD, roles, branch assignment
branches	CRUD, franchise toggle
inventory	Drugs/batches CRUD, transfer, stats, expiring
sales	Batch sale, summary, list, mark credit paid
settings	Franchise amount; integration_settings KV secrets
integrations/titak	Price pull → update drug batches sellingPrice
integrations/pos	Initiate / confirm / status terminal sessions
integrations/insurance	Adapter registry validate/submit hooks
loss-reports	Submit and review workflow
reporting	Aggregated sales; CSV/PDF exporters
purchasing	Suppliers, POs, OCR client

6.3 Concurrency

Sales and transfers open TypeORM transactions and acquire **pessimistic_write** locks on DrugBatch rows before decrementing stock. Insufficient stock or cross-branch batch misuse raises operational AppErrors.

7. Frontend Architecture & UX Design

7.1 Design system

Visual language targets a professional clinical aesthetic: medical teal primary, restrained gold accent, slate typography, and glass panels (.glass, .glass-strong, backdrop blur). Components share tokens for light and dark themes.

7.2 Navigation & roles

Sidebar entries are filtered by role helpers. Dashboard KPIs use dedicated summary endpoints so totals are independent of table page size.

7.3 Internationalization & theme

Locales **en** and **fa** are always prefixed in the URL. LanguageSwitcher and ThemeToggle appear on login and setup.

7.4 Client data layer

lib/api.ts centralizes Axios (base URL, 30s timeout, Bearer token, outbound x-request-id). useApi wraps calls with toast mapping; 401 triggers session clear and redirect.

7.5 Key screens

- Login / Setup — glass card, theme + language controls
- Dashboard — KPIs (total drugs, today's sales IRR, expiry, low stock, pending loss)
- Inventory drugs & batches — catalog, Titak update, transfers
- Sales — multi-tab baskets, insurance, POS panel, IRR totals
- Sales records & credits — filters, pagination, mark paid
- Reports — date/branch filters, CSV/PDF export
- Settings — franchise, integration keys, password change

8. Data Model & Persistence

Entity	Notable fields
Employee	email, passwordHash, role, currentBranchId
EmployeeSession	login/logout times, IP
Branch	name, address, isWarehouse, hasFranchise
Drug	name, brand, company, titakCode, insuranceEligible, insuranceCode
DrugBatch	drugId, branchId, expirationDate, count, prices, isOffer
StockMovement	type, quantity, branches, performer
SaleTransaction	prices, paymentMethod, insurance*, patientShare, basketId, isPaid
Settings	key + numeric value (franchise_amount)
IntegrationSetting	string key/value secrets and config
LossReport	status pending/approved/rejected
Supplier / PurchaseOrder	purchasing domain

Compose defaults TYPEORM_SYNCHRONIZE=false. For first bootstrap set true temporarily or run migrations. Production should use explicit migrations.

9. Security & Access Control

9.1 Authentication

Passwords hashed with bcrypt. Login issues JWT. Optional Redis-backed rate limiting.

9.2 Authorization

RBAC middleware restricts routes. Sales listing for non-manager roles is scoped to current employee/branch. Integration secrets are masked in list APIs.

9.3 Hardening

- Helmet HTTP headers
- Zod input validation on write endpoints
- Secrets via environment and integration_settings
- Database and Redis not published on host by default
- JSON body size limit (1mb)

10. Sales, Insurance & POS Logic

10.1 Basket sale algorithm

POST /sales/batch accepts items and payment. Inside a transaction, each batch is locked; stock validated and decremented; SaleTransaction and StockMovement inserted. Optional franchise fee on first line when enabled.

10.2 Insurance share rules

- insuranceProvider none → full patient share
- Provider set → insuranceMemberId required
- Coverage % from insurance_default_coverage_percent (default 70)
- Only insuranceEligible drugs receive insurer share
- Line stores coverage and patient share amounts

10.3 POS state machine

```
idle → initiate(amount=patientShare) → pending
pending → confirm(true) → approved → complete sale + posReference
pending → confirm(false) → failed → retry
```

11. Integrations

11.1 Titak

Key from IntegrationSetting then env. Base URL https://api.titak.ir/v1. External id = titakCode. Updates lastPriceUpdateDate and batch sellingPrice.

11.2 Insurance APIs

Tamin/Salamat/Mosalah keys in settings. Live claim adapters are contract-based; software does not ship third-party secrets.

11.3 POS

Routes: /integrations/pos/initiate, /confirm, /status/:referenceCode. Amounts in whole IRR.

12. Error Handling & Observability

```
{ "success": false, "code": "NOT_FOUND", "message": "…", "requestId": "uuid" }
```

Source	HTTP	code
ZodError	400	VALIDATION_ERROR
AppError	varies	err.code
QueryFailedError	400	DATABASE_ERROR
JWT errors	401	UNAUTHORIZED
Malformed JSON	400	BAD_REQUEST
Unknown	500	INTERNAL_ERROR

Correlate toast ref / x-request-id with backend logs. See docs/ERROR_HANDLING_AND_LOGGING.md.

13. Infrastructure & Deployment

Service	Image / build	Notes
postgres	postgres:16-alpine	Volume postgres_data; healthcheck
redis	redis:7-alpine	AOF; 64mb maxmemory LRU
backend	../backend Dockerfile	Depends on DB+Redis healthy
frontend	../frontend Dockerfile	NEXT_PUBLIC_API_URL=/api/v1
nginx	nginx:1.27-alpine	Only published port HTTP_PORT

```
cd infrastructure && cp .env.example .env
docker compose up --build -d
docker compose up -d --build backend frontend
docker compose logs -f backend
```

Also supports Oracle Cloud Always Free ARM — see infrastructure/deploy/oracle-cloud.md.

14. API Reference Summary

Base prefix: `/api/v1`.

Area	Representative paths
Setup	POST /setup
Auth	POST /auth/login, GET /auth/me, PUT /auth/change-password
Employees	GET/POST /employees, GET/PUT /employees/:id
Branches	GET/POST /branches, PATCH /branches/:id/franchise
Inventory	CRUD drugs/batches, transfer, catalog/stats
Sales	GET /sales, /summary, POST /batch, basket pay
POS	POST initiate|confirm, GET status/:ref
Titak	POST update-price/:drugId
Settings	GET/PUT franchise; GET/PUT integrations
Reporting	GET sales, export CSV/PDF
Loss reports	GET/POST, review
Health	GET /health

15. Configuration & Environment

Variable	Where	Purpose
POSTGRES_*	Compose .env	Database credentials and name
JWT_SECRET	Compose / backend	Token signing (required in prod)
JWT_EXPIRES_IN	Backend	e.g. 7d
REDIS_URL	Backend	Rate limit store
TYPEORM_SYNCHRONIZE	Backend	Schema auto-sync (bootstrap only)
TITAK_API_KEY	Backend env	Fallback if not in Settings UI
LOG_LEVEL	Backend	debug|info|warn|error
LOG_TO_FILES	Backend	Optional file transports
NEXT_PUBLIC_API_URL	Frontend	/api/v1 behind Nginx
HTTP_PORT	Compose	Host port mapped to Nginx

16. Operational Runbook

16.1 First boot

- Configure .env secrets
- Bring stack up; confirm /health returns ok
- Open /en/setup or /fa/setup; create manager
- Create branches; seed drugs and batches
- Configure Titak/insurance keys under Settings if available

16.2 Incident correlation

Collect requestId from UI toast or x-request-id; search backend logs. Check Postgres and Redis. For stock issues review StockMovement and concurrent sale errors.

16.3 Backup

Persist volume postgres_data; schedule pg_dump in production.

17. Testing Strategy

Backend Jest: services (sales, inventory transfer, auth, employees, settings, loss-reports), middleware (auth, rbac, validation, rateLimit), utilities (pagination, JWT, AppError). Frontend: utils, navigation, useRole, ErrorContext, ApiError. Extend service-level tests when changing domain rules (insurance, POS, stock).

18. Roadmap & Extension Points

- Production TypeORM migrations for all schema changes
- Redis-backed POS session store for multi-replica backends
- Real BehMellat / acquirer SDK wiring
- Live insurer claim adapters using stored API keys
- TLS termination and hardened CORS_ORIGIN
- Optional Sentry sink behind clientLog / Winston
- E2E tests (Playwright) for basket + POS + insurance paths

19. Glossary

Term	Meaning
IRR	Iranian Rial — retail display/storage currency
Batch	Stock lot of a drug at a branch with shared expiry and prices
Basket	Grouped sale lines for one customer interaction
Franchise fee	Optional fixed fee when branch.hasFranchise
Titak	External drug price information service
Tamin / Salamat / Mosalah	Major Iranian insurance funds
RBAC	Role-based access control
requestId	Correlation id across client and API logs
AppError	Operational backend error safe for clients
POS	Card terminal payment flow

End of catalog — Pharmacy Management System. Source: github.com/yasin6606/pharmacy-management-system. Reflects system as of 2026-08-19.